Hello class,

My name is Zachery Thompson, although I typically go by Zac. I am married, have a six-month-old daughter, and currently teach middle school band, orchestra, and choir. I have been programming since high school, where I took AP Computer Science, and I continued studying programming during my undergraduate degree while preparing to become a teacher. Since then, programming has become one of my favorite ways to solve practical problems. I have developed an inventory application for my school district to help manage our instrument inventory, and I have also built applications that simulate tabletop role-playing game (TTRPG) combat so I can more accurately evaluate encounter difficulty when designing adventures for my players. I am most familiar with Python, Java, JavaScript, TypeScript, HTML, and CSS, as well as frameworks such as Django, Spring Boot, and Vue. I have also worked with front-end libraries such as Bootstrap and Tailwind CSS. I am looking forward to learning alongside everyone this term and seeing how we can continue improving our software design skills.

Two of the foundational concepts of object-oriented programming are **encapsulation** and **inheritance**. Encapsulation is the practice of combining data and the methods that operate on that data into a single class while restricting direct access to an object's internal state. Instead of allowing outside code to freely modify variables, access is controlled through methods such as getters and setters, which can perform validation before data is stored (Agarwal, 2026). This improves security, maintainability, and modularity because changes to the internal implementation typically do not affect other parts of an application. One disadvantage is that encapsulation can require additional code and introduce a small amount of overhead, but the benefits usually outweigh these costs. Inheritance, on the other hand, allows one class to inherit

properties and behaviors from another, reducing duplicate code and creating logical class hierarchies (Kartik, 2026). A simple example would be an Animal superclass that defines common behaviors, while subclasses such as Horse and Wolf inherit those shared characteristics but implement species-specific behaviors such as different sounds, movement, or feeding methods. During application development, both concepts have practical value. In an instrument inventory system, encapsulation could protect student or inventory records by preventing unauthorized or invalid modifications while allowing approved methods to validate changes before updating the data. Inheritance could model different instrument categories, with a general Instrument class providing common attributes, followed by Woodwind, Brass, and Percussion subclasses, and individual instruments such as Clarinet, Trumpet, or SnareDrum inheriting those shared characteristics. Similarly, a user management system could include a general User class with subclasses such as Teacher, Student, and DepartmentChair, where the department chair inherits all teacher functionality while adding administrative responsibilities.

These principles improve software by making code easier to understand, maintain, and extend. Encapsulation promotes safer code because data is accessed through well-defined interfaces rather than manipulated directly, reducing the likelihood of invalid states or unintended side effects (Agarwal, 2026). Inheritance reduces duplicated code by allowing developers to reuse existing functionality while extending it when needed, leading to cleaner and more maintainable applications (Kartik, 2026). I have experienced these benefits firsthand while developing software projects outside of the classroom. Organizing related functionality into reusable classes has made my projects significantly easier to expand as new requirements emerged. Rather than rewriting existing logic, I have often been able to extend previous work, saving time and making future maintenance much more manageable. I expect these object-

oriented principles will continue to become even more valuable as we develop larger and more complex Java applications throughout this course.

**References**

Agarwal, H. (2026, April 23). *Encapsulation in Java*. GeeksforGeeks. https://www.geeksforgeeks.org/java/encapsulation-in-java/

Kartik. (2026, June 13). *Inheritance in Java*. GeeksforGeeks. https://www.geeksforgeeks.org/java/inheritance-in-java/